

Workshop 01 – Erstes CNN zur Bilderkennung mit Keras

Ziel

Entwickle ein KI-Modell zur Bilderkennung mit der Keras-Bibliothek in Python. Das Modell soll Convolutional Layers (Conv2D) und Dense Layers enthalten. Plote die Trainings- und Validierungsgenauigkeit sowie den Verlust, um den Trainingsverlauf zu visualisieren. Versuche anschließend, verschiedene Hyperparameter zu optimieren, um die Leistung des Modells zu verbessern.

Datensatz

Verwende den **CIFAR-10**-Datensatz. Er umfasst 60.000 Farbbilder (32×32 Pixel) in 10 Klassen mit jeweils 6.000 Bildern und ist direkt über Keras zugänglich:

```
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

Aufgaben

Arbeite die Punkte in `workshop.py` ab. Die Stellen sind mit `TODO` markiert.

1. Daten laden und vorbereiten

- Lade den CIFAR-10-Datensatz über Keras.
- Bereite die Daten für das Training vor (Normalisierung der Pixelwerte auf den Bereich 0–1, Aufteilung in Trainings- und Testdaten).

2. Modell erstellen – ein neuronales Netz mit

- mindestens zwei Convolutional Layers (Conv2D) mit geeigneten Hyperparametern,
- einem Pooling Layer nach jedem Convolutional Layer,
- mindestens zwei Dense Layers für die Klassifikation.

3. Modell kompilieren und trainieren

- Kompiliere das Modell mit einem geeigneten Optimizer und einer passenden Verlustfunktion.
- Trainiere das Modell mit den Trainingsdaten und nutze die Testdaten zur Validierung.

4. Trainingsverlauf visualisieren

- Plote `accuracy / val_accuracy` und `loss / val_loss` über die Epochen mit Matplotlib und speichere den Plot als `plot.png`.

5. Evaluierung und Ergebnisanalyse

- Bewerte die Leistung des Modells anhand der Testdaten und analysiere die Ergebnisse: Wo liegt die Test-Accuracy? Gibt es Anzeichen für Overfitting?

Bonus (freiwillig)

- Experimentiere mit Hyperparametern (Anzahl Filter, Kernelgröße, Epochen, Dense-Breite, Dropout) und vergleiche die Ergebnisse.
- Logge das Training mit dem `TensorBoard`-Callback und schau dir den Verlauf in TensorBoard an.

Hinweise

- Als Verlustfunktion bietet sich `sparse_categorical_crossentropy` an, da die Labels als Integer (0–9) vorliegen.
- 10 Epochen reichen für einen ersten Eindruck; auf CPU dauert das wenige Minuten.